In God we trust

نام و نام خانوادگی : مهدیس میرزایی

شماره دانشجویی : ۸۱۰۱۰۲۵۵۰

پروژه : پروژه شماره 6

تمرین یک :

بخش ۱ و ۲) در این بخش، پارامترهای اولیه رادار تعریف شده و سیگنال‌های ارسالی و دریافتی مدل‌سازی می‌شوند.

```
clc; clear; close all;

f_base = 5;
s_rate = 100;
span = 1;
ticks = 0:1/s_rate:span-1/s_rate;
c_vel = 3e8;

dist_val = 250e3;
v_val_kmh = 180;
v_val_ms = v_val_kmh / 3.6;
att_fact = 0.5;
dop_sens = 0.3;

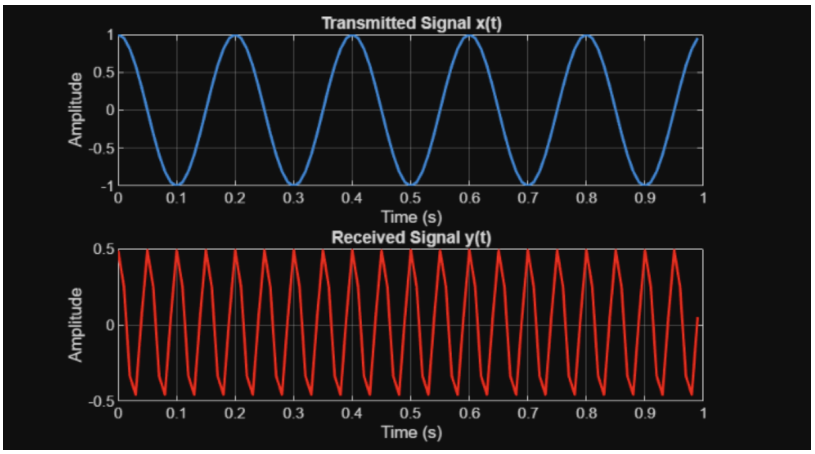
f_dop = dop_sens * v_val_ms;
t_delay = 2 * dist_val / c_vel;

sig_tx = cos(2*pi*f_base*ticks);

figure;
subplot(2,1,1);
plot(ticks, sig_tx, 'LineWidth', 1.5);
title('Transmitted Signal x(t)');
xlabel('Time (s)'); ylabel('Amplitude');
grid on;

sig_rx = att_fact * cos(2*pi*(f_base + f_dop)*(ticks - t_delay));

subplot(2,1,2);
plot(ticks, sig_rx, 'r', 'LineWidth', 1.5);
title('Received Signal y(t)');
xlabel('Time (s)'); ylabel('Amplitude');
grid on;
```



بخش۳) با استفاده از تبدیل فوریه (FFT)، فرکانس غالب و فاز سیگنال برای استخراج سرعت و فاصله محاسبه می‌شود.

```
len_sig = length(sig_rx);
spec_rx = fftshift(fft(sig_rx));
freq_bins = (-len_sig/2 : len_sig/2 - 1) * (s_rate/len_sig);

[~, peak_pos] = max(abs(spec_rx));
f_found = freq_bins(peak_pos);

if f_found < 0
    f_found = -f_found;
    phase_val = -angle(spec_rx(peak_pos));
else
    phase_val = angle(spec_rx(peak_pos));
end

f_dop_est = f_found - f_base;
v_est_val = f_dop_est / dop_sens * 3.6;
t_delay_est = -phase_val / (2*pi*f_found);

while t_delay_est < 0
    t_delay_est = t_delay_est + 1/f_found;
end

dist_est_val = (c_vel * t_delay_est) / 2 / 1000;

fprintf('Estimated Speed: %.2f Km/h (Actual: %.2f)\n', v_est_val, v_val_kmh);
fprintf('Estimated Distance: %.2f Km (Actual: %.2f)\n', dist_est_val,
dist_val/1000);
```

```
Estimated Speed: 180.00 Km/h (Actual: 180.00)
Estimated Distance: 250.00 Km (Actual: 250.00)
```

که طبق خروجی سرعت ۱۸۰km/h و فاصله 250km است.

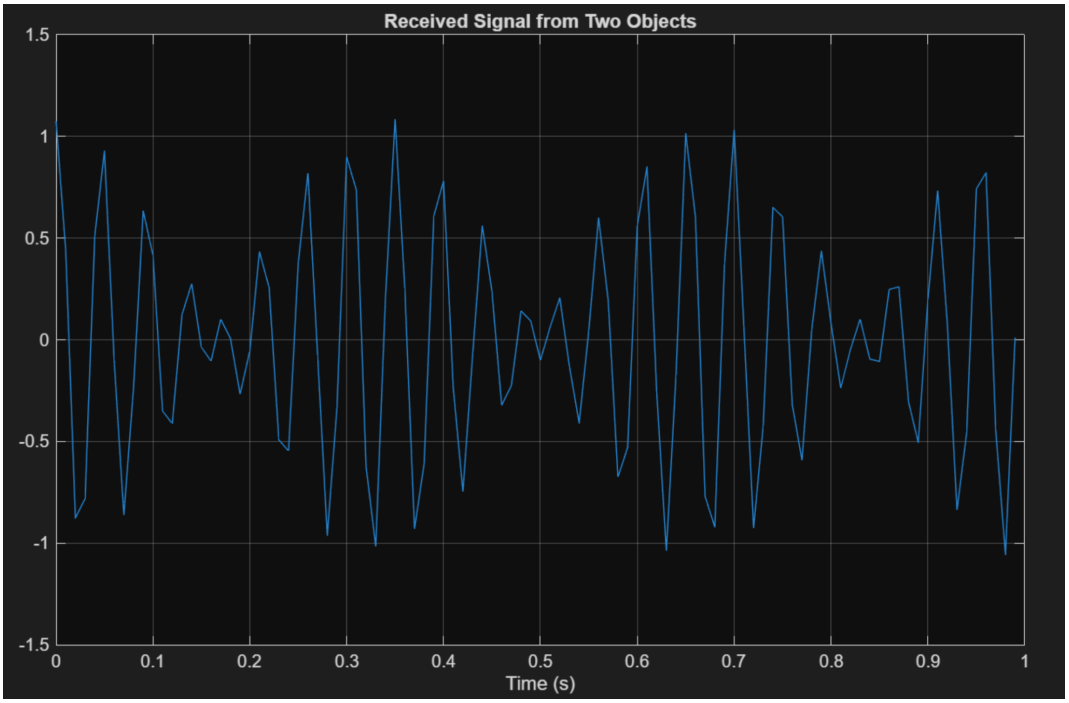
بخش ۴) در این بخش، تأثیر نویز با دامنه‌های مختلف بر دقت تخمین بررسی می‌کنیم.

```
levels = 0:0.5:15;
fprintf('\nNoise Amp | Est Speed (Km/h) | Est Dist (Km) | Status\n');
fprintf('-------------------------------------------------------------\n');
for n_amp = levels
    interference = n_amp * (rand(size(sig_rx)) - 0.5);
    sig_dirty = sig_rx + interference;
    n_pts = length(sig_dirty);
    n_spec = fftshift(fft(sig_dirty));
    n_freqs = (-n_pts/2 : n_pts/2 - 1) * (s_rate/n_pts);
    [~, n_peak_idx] = max(abs(n_spec));
    n_f_peak = n_freqs(n_peak_idx);
    if n_f_peak < 0
        n_f_peak = -n_f_peak;
        n_phase = -angle(n_spec(n_peak_idx));
    else
        n_phase = angle(n_spec(n_peak_idx));
    end
    n_dop_est = n_f_peak - f_base;
    n_v_est = n_dop_est / dop_sens * 3.6;
    n_t_est = -n_phase / (2*pi*n_f_peak);
    while n_t_est < 0
        n_t_est = n_t_est + 1/n_f_peak;
    end
    n_dist_est = (c_vel * n_t_est) / 2 / 1000;
    if abs(n_v_est - v_val_kmh) > (0.05 * v_val_kmh)
        state = '!!! SPEED FAILED !!!';
    elseif abs(n_dist_est - dist_val/1000) > (0.05 * dist_val/1000)
        state = 'DIST WRONG';
    else
        state = 'OK';
    end
    fprintf('%9.1f | %16.2f | %13.2f | %s\n', n_amp, n_v_est, n_dist_est,
state);
end
```

```
Noise Amp | Est Speed (Km/h) | Est Dist (Km) | Status
-------------------------------------------------------------
      0.0 |           180.00 |        250.00 | OK
      0.5 |           180.00 |        260.38 | OK
      1.0 |           180.00 |         78.25 | DIST WRONG
      1.5 |           180.00 |        597.10 | DIST WRONG
      2.0 |           180.00 |        321.17 | DIST WRONG
      2.5 |           180.00 |        177.90 | DIST WRONG
      3.0 |           180.00 |        220.01 | DIST WRONG
      3.5 |           180.00 |        278.14 | DIST WRONG
      4.0 |           216.00 |       5803.77 | !!! SPEED FAILED !!!
      4.5 |           516.00 |       2433.59 | !!! SPEED FAILED !!!
      5.0 |           468.00 |       1605.76 | !!! SPEED FAILED !!!
      5.5 |           -12.00 |      33069.30 | !!! SPEED FAILED !!!
      6.0 |           -48.00 |      73351.61 | !!! SPEED FAILED !!!
      6.5 |           108.00 |      10526.19 | !!! SPEED FAILED !!!
      7.0 |           -12.00 |        545.23 | !!! SPEED FAILED !!!
      7.5 |           312.00 |        490.12 | !!! SPEED FAILED !!!
      8.0 |           540.00 |       1500.00 | !!! SPEED FAILED !!!
      8.5 |           300.00 |       4260.59 | !!! SPEED FAILED !!!
      9.0 |           492.00 |       3120.93 | !!! SPEED FAILED !!!
      9.5 |           372.00 |       2516.79 | !!! SPEED FAILED !!!
     10.0 |           372.00 |       3120.68 | !!! SPEED FAILED !!!
     10.5 |           372.00 |       1532.16 | !!! SPEED FAILED !!!
     11.0 |           156.00 |       3029.41 | !!! SPEED FAILED !!!
     11.5 |            48.00 |       7264.88 | !!! SPEED FAILED !!!
     12.0 |           432.00 |       2184.75 | !!! SPEED FAILED !!!
     12.5 |           540.00 |       1500.00 | !!! SPEED FAILED !!!
     13.0 |           540.00 |       1500.00 | !!! SPEED FAILED !!!
     13.5 |            36.00 |        385.46 | !!! SPEED FAILED !!!
     14.0 |           384.00 |       3563.68 | !!! SPEED FAILED !!!
     14.5 |           -48.00 |     135067.84 | !!! SPEED FAILED !!!
     15.0 |           -24.00 |      28458.95 | !!! SPEED FAILED !!!
```

بخش ۵ و ۶) در این بخش سیگنال‌های بازگشتی از دو هدف با سرعت‌ها و فاصله‌های متفاوت ترکیب شده و شناسایی می‌شوند.

```
range1 = 250e3; v1_raw = 180/3.6; g1 = 0.5;
range2 = 200e3; v2_raw = 216/3.6; g2 = 0.6;
dop1 = dop_sens * v1_raw; dly1 = 2 * range1 / c_vel;
dop2 = dop_sens * v2_raw; dly2 = 2 * range2 / c_vel;
part1 = g1 * cos(2*pi*(f_base + dop1)*(ticks - dly1));
part2 = g2 * cos(2*pi*(f_base + dop2)*(ticks - dly2));
total_sig = part1 + part2;
figure;
plot(ticks, total_sig);
title('Received Signal from Two Objects');
xlabel('Time (s)'); grid on;
total_spec = fftshift(fft(total_sig));
total_mag = abs(total_spec);
limit = max(total_mag) * 0.5;
[~, loc_points] = findpeaks(total_mag, 'MinPeakHeight', limit);
valid_locs = loc_points(freq_bins(loc_points) > 0);
found_freqs = freq_bins(valid_locs);
found_phases = angle(total_spec(valid_locs));
for m = 1:length(found_freqs)
    curr_f = found_freqs(m);
    curr_phi = found_phases(m);
    curr_v = (curr_f - f_base) / dop_sens * 3.6;
    curr_t = -curr_phi / (2*pi*curr_f);
    while curr_t < 0
        curr_t = curr_t + 1/curr_f;
    end
    curr_r = (c_vel * curr_t) / 2 / 1000;
    fprintf('Object %d detected: V = %.2f Km/h, R = %.2f Km\n', m, curr_v,
curr_r);
end
```




```
Object 1 detected: V = 180.00 Km/h, R = 250.00 Km
Object 2 detected: V = 216.00 Km/h, R = 200.00 Km
```

بخش ۷) اگر سرعت دو جسم یکسان باشد، فرکانس‌های داپلر آن‌ها ادغام شده و هر دو در طیف فرکانسی به صورت یک قله (Peak) واحد ظاهر می‌شوند که تشخیص آن‌ها را با روش‌های ساده دشوار می‌کند.

بخش ۸) در صورتی که فاصله‌ها برابر اما سرعت‌ها متفاوت باشند، فرکانس‌های داپلر متمایز باقی می‌مانند و دو قله جداگانه در حوزه فرکانس به راحتی قابل شناسایی و تخمین خواهند بود

بخش ۹) برای تشخیص تعداد نامعلوم اجسام، می‌توان از الگوریتم‌های یافتن قله (Peak Finding) به همراه یک حد آستانه (Threshold) برای حذف نویز استفاده کرد تا تعداد قله‌های معنادار معادل تعداد اجسام به دست آید

تمرین دو :

بخش ۱) با توجه به نت‌های داده شده کد زیر اجرا شد و آهنگ تیلور بود 3>

```
fs = 8000;
T = 0.5;
T_half = 0.25;
tau = 0.025;
fD = 587.33;
fE = 659.25;
fFs = 739.99;
fG = 783.99;
t_L = 0:1/fs:T-1/fs;
t_S = 0:1/fs:T_half-1/fs;
t_R = 0:1/fs:tau-1/fs;
rest = zeros(size(t_R));
note = @(freq, time) sin(2*pi*freq*time);

m1 = [note(fD,t_S), rest, note(fD,t_S), rest, note(fG,t_L), rest, note(fFs,t_L),
rest, note(fD,t_L), rest];
m2 = [note(fD,t_S), rest, note(fE,t_S), rest, note(fE,t_S), rest,note(fD,t_S),
rest, note(fFs,t_S), rest,note(fD,t_S), rest, note(fE,t_L), rest];
m3 = [note(fD,t_L), rest, note(fE,t_L), rest, note(fFs,t_L), rest, note(fE,t_L),
rest];
m4 = m2;

m5 = [note(fD,t_L), rest, note(fE,t_S), rest, note(fD,t_S), rest, note(fFs,t_L),
rest, note(fE,t_L), rest];
m6 = m5;
m7 = [note(fD,t_S), rest, note(fD,t_S), rest, note(fE, t_L), rest, note(fFs,
t_S), rest, note(fE, t_S), rest, note(fFs, t_L), rest];
m8 = [note(fFs,t_S), rest, note(fE,t_S), rest, note(fFs,t_L), rest,
note(fFs,t_L), rest, note(fD,t_L), rest];

song = [m1, m2, m3, m4, m5, m6, m7, m8];
sound(song, fs);
audiowrite('my_song.wav', song, fs);
```

بخش ۲) برای ترانه fly me to the moon این کار را انجام می‌دهیم. در واقع نت‌های آن را با توابع سینوسی می‌سازیم و در فرمت wav ذخیره می‌کنیم و طبق خروجی تعداد بیت در هر سمپل ۱۶ تا بود.

```
rate = 8000;
len_full = 0.5;
len_half = 0.25;
gap = 0.025;
f_C = 523.25; f_B = 493.88; f_A = 440.00; f_G = 392.00; f_F = 349.23;
f_E = 329.63; f_D = 293.66;

t_f = 0:1/rate:len_full-1/rate;
t_h = 0:1/rate:len_half-1/rate;
t_g = 0:1/rate:gap-1/rate;

silence = zeros(size(t_g));
play = @(frq, tm) sin(2*pi*frq*tm);

ph1 = [play(f_C, t_f), silence, play(f_B, t_h), silence, play(f_A, t_h),
silence, play(f_G, t_h), silence, play(f_F, t_f), silence];
ph2 = [play(f_G, t_h), silence, play(f_A, t_h), silence, play(f_C, t_f),
silence, play(f_B, t_h), silence, play(f_A, t_h), silence, play(f_G, t_h),
silence, play(f_F, t_f), silence, play(f_E, t_f)];
final_melody = [ph1, ph2];
sound(final_melody, rate);
audiowrite('moon_song.wav', final_melody, rate);
info = audioinfo('moon_song.wav');
fprintf('Bits per sample: %d\n', info.BitsPerSample);
```

بخش ۳) با تقسیم سیگنال به پنجره‌های زمانی کوچک و اعمال تبدیل فوریه، فرکانس‌های موجود شناسایی و به نام نت‌ها تبدیل می‌شوند.

```
function [note_list, time_list] = extract_notes(song, fs)
    pitches = [523.25, 554.37, 587.33, 622.25, 659.25, 698.46, 739.99, 783.99,
830.61, 880.00, 932.33, 987.77, 0];
    labels = {'C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B',
'Rest'};
    win_len = floor(0.05 * fs);
    note_list = {};
    time_list = [];
    prev_pitch = -1;
    for k = 1:win_len:(length(song) - win_len)
        block = song(k : k + win_len);
        if max(abs(block)) < 0.01
            detected_f = 0;
        else
            L = length(block);
            Y_val = fft(block);
            mag = abs(Y_val(1:floor(L/2)+1));
            f_vec = (0:floor(L/2)) * (fs/L);
            [~, peak_idx] = max(mag);
            detected_f = f_vec(peak_idx);
        end
        [~, best_fit] = min(abs(pitches - detected_f));
        match_f = pitches(best_fit);
        match_name = labels{best_fit};
        if match_f ~= prev_pitch
            note_list{end+1} = match_name;
            time_list(end+1) = win_len/fs;
            prev_pitch = match_f;
        else
            time_list(end) = time_list(end) + win_len/fs;
        end
    end
end

[track, track_fs] = audioread('my_song.wav');
[found_notes, found_times] = extract_notes(track, track_fs);
output_table = table(found_notes', found_times', 'VariableNames', {'Note',
'Seconds'});
disp(output_table);
```

Note	Seconds
{'D' }	0.55
{'G' }	0.5
{'F#'}	0.55
{'D' }	0.8
{'E' }	0.55
{'D' }	0.25
{'F#'}	0.3
{'D' }	0.25
{'E' }	0.55
{'D' }	0.5
{'E' }	0.55
{'F#'}	0.5
{'E' }	0.55
{'D' }	0.25
{'E' }	0.55
{'D' }	0.3
{'F#'}	0.25
{'D' }	0.3
{'E' }	0.5
{'D' }	0.55
{'E' }	0.25
{'D' }	0.3
{'F#'}	0.5
{'E' }	0.55
{'D' }	0.5
{'E' }	0.3
{'D' }	0.25
{'F#'}	0.55
{'E' }	0.5
{'D' }	0.55
{'E' }	0.55
{'F#'}	0.25
{'E' }	0.3
{'F#'}	0.8
{'E' }	0.25
{'F#'}	1.05
{'D' }	0.5